

```
In [ ]: import marimo as mo
import pandas as pd
from ortools.sat.python import cp_model
import time
```

Função `cria_variaveis()`

A função `cria_variaveis()` cria as **variáveis de decisão** do horário.

Para cada combinação de **turma, disciplina, dia e período**, é criada uma variável com valor `0` ou `1`:

- `0` → a aula não acontece nesse horário.
- `1` → a aula acontece nesse horário.

As variáveis são guardadas num dicionário para serem usadas posteriormente pelas regras **R1-R7** e pelo solver.

Por exemplo:

`variaveis[("7ºA", "Matemática", "Seg", 2)]`
representa se a turma 7ºA tem Matemática na segunda-feira, período 2.

```
In [ ]: def cria_variaveis(modelo, turma, disciplinas, dia, periodo):
    variaveis = {}
    for t in turma:
        for c in range(len(disciplinas)):
            for d in dia:
                for p in periodo:
                    nome = f"{t}_{disciplinas['disciplina'][c]}_{d}_{p}"
                    variaveis[(t, disciplinas['disciplina'][c], d, p)] =
modelo.NewIntVar(0, 1, nome)
    return variaveis
```

Função `aplica_regras()`

A função `aplica_regras()` aplica ao modelo as **restrições que garantem que o horário é válido**.

Cada variável $x_{t,d,c,p}$ representa se uma disciplina é dada por uma turma num determinado dia e período.

R1 — Uma aula por turma e período

Para cada turma, dia e período, a soma das disciplinas não pode ser superior a 1:

$$\sum_c x_{t,c,d,p} \leq 1$$

Ou seja, uma turma não pode ter duas disciplinas ao mesmo tempo.

R2 — Carga semanal

Cada disciplina tem de cumprir exatamente o número de aulas definido:

$$\sum_d \sum_p x_{t,c,d,p} = \text{carga_semanal}_c$$

R3 — Disciplina simples

Uma disciplina que não é de período duplo só pode aparecer **uma vez por dia**:

$$\sum_p x_{t,c,d,p} \leq 1$$

R4 — Período duplo

Quando uma disciplina é de período duplo, as duas aulas têm de ocupar **períodos consecutivos** no mesmo dia, ou seja, seguindo um dos padrões:

(1,1,0,0,0) ou (0,1,1,0,0) ou (0,0,1,1,0) ou (0,0,0,1,1)

R5 — Conflito de professores

Um professor não pode dar aulas a duas turmas no mesmo período:

$$\sum_t \sum_c x_{t,c,d,p} \leq 1$$

considerando apenas as disciplinas lecionadas por esse professor.

R6 — Indisponibilidade

Quando um professor está indisponível num determinado dia e período, a aula é obrigatoriamente proibida:

$$x_{t,c,d,p} = 0$$

R7 — Capacidade das salas

O número de aulas que necessitam de uma determinada sala não pode ultrapassar a sua capacidade:

$$\sum_{t,c} x_{t,c,d,p} \leq \text{capacidade da sala}$$

Estratégia de otimização dos buracos

Este código calcula os **buracos no horário de cada professor em cada dia**, para que possam ser minimizados pelo solver.

Primeiro, para cada professor e dia, é criada uma variável `presenca` para cada período:

$$presenca_p = \sum x_{t,c,d,p}$$

onde `presenca = 1` significa que o professor tem pelo menos uma aula nesse período.

Depois são identificados:

$$\text{primeiro_tempo} = \min(p)$$

e

$$\text{ultimo_tempo} = \max(p)$$

considerando apenas os períodos em que o professor tem aulas.

O número de períodos entre a primeira e a última aula é:

$$\text{ultimo_tempo} - \text{primeiro_tempo} + 1$$

Para calcular os buracos, subtrai-se o número total de aulas desse intervalo:

$$\text{buracos_dia} = (\text{ultimo_tempo} - \text{primeiro_tempo} + 1) - \text{total_aulas_dia}$$

Por exemplo, se um professor tiver aulas nos períodos **1, 2, 4 e 5**:

$$(5 - 1 + 1) - 4 = 1$$

Existe, portanto, **1 buraco**, que corresponde ao período 3.

Se o professor não tiver nenhuma aula nesse dia:

$$\text{buracos_dia} = 0$$

No final, todos os valores são guardados em `buracos_totais`. O objetivo do modelo pode então **minimizar a soma destes buracos**, procurando concentrar as aulas dos professores e evitar períodos livres entre aulas.

Penalização do último período

Para evitar aulas no último período do dia, conta-se a presença de cada professor nesse período.

$$\text{Penalização} = \sum_{prof, dia} Presença_{prof, dia, 5}$$

onde:

$$Presença = \begin{cases} 1 & \text{se o professor tem aula no período 5} \\ 0 & \text{caso contrário} \end{cases}$$

Esta soma é incluída na função objetivo e **minimizada pelo solver**, fazendo com que sejam preferidos horários com menos professores a lecionar no último período.

Assim, a função `aplica_regras()` transforma as regras do problema em **restrições matemáticas que o solver CP-SAT utiliza para construir um horário válido**.

```
In [ ]: def
aplica_regras(variaveis, turma, dia, periodo, turmas, salas, disciplinas, disponib
ilidade_execoes, modelo):
    # R1
    for t in turma:
        for d in dia:
            for p in periodo:

                aulas_no_tempo = [variaveis[(t, disciplinas['disciplina']
[c], d, p)] for c in range(len(disciplinas))]
                modelo.Add(sum(aulas_no_tempo) <= 1)

    # R2
    for t in turma:
        for c in range(len(disciplinas)):
            disc = disciplinas['disciplina'][c]
            carga = int(disciplinas['carga_semanal'][c])
            soma_semanal = sum(variaveis[(t, disc, d, p)] for d in dia for
p in periodo)
            modelo.Add(soma_semanal == carga)

    for t in turma:
        for c in range(len(disciplinas)):
            disciplina_analizar = disciplinas["disciplina"][c]
            e_duplo = str(disciplinas["duplo_periodo"][c]).strip().lower()
            == "sim"

            for d in dia:
                vars_dia = [variaveis[(t, disciplina_analizar, d, p)] for p
in periodo]

                if not e_duplo:
                    modelo.Add(sum(vars_dia) <= 1)
                else:

                    padroes_validos = [
```

```

        [0, 0, 0, 0, 0], # Zero aulas
        [1, 1, 0, 0, 0], # Aulas nos tempos 1 e 2
        [0, 1, 1, 0, 0], # Aulas nos tempos 2 e 3
        [0, 0, 1, 1, 0], # Aulas nos tempos 3 e 4
        [0, 0, 0, 1, 1] # Aulas nos tempos 4 e 5
    ]
    modelo.AddAllowedAssignments(vars_dia, padroes_validos)

disciplinas_por_professor = {}

for i in range(len(disciplinas)):
    prof = disciplinas["professor"][i]
    disc = disciplinas["disciplina"][i]

    if prof not in disciplinas_por_professor:
        disciplinas_por_professor[prof] = []

    disciplinas_por_professor[prof].append(disc)

# R5
for prof, discs in disciplinas_por_professor.items():
    for d in dia:
        for p in periodo:
            aulas_prof = []
            for disc in discs:
                for t in turma:
                    aulas_prof.append(variaveis[(t, disc, d, p)])
            modelo.Add(sum(aulas_prof) <= 1)

#R6
for i in range(len(disponibilidade_execoes)):
    # Lemos o professor, dia e período do DataFrame
    prof_indisponivel = str(disponibilidade_execoes["professor"]
[i]).strip()
    dia_indisp = str(disponibilidade_execoes["dia"][i]).strip()
    periodo_indisp = int(disponibilidade_execoes["periodo"][i])

    # 1. Procurar nas disciplinas quais são dadas por este professor
    for c in range(len(disciplinas)):
        prof_da_disciplina = str(disciplinas["professor"][c]).strip()

        if prof_da_disciplina == prof_indisponivel:
            disciplina_do_prof = disciplinas["disciplina"][c]

            # 2. Bloquear esta aula para TODAS as turmas nesse tempo
            for t in turma:
                v = variaveis[(t, disciplina_do_prof, dia_indisp,
periodo_indisp)]
                modelo.Add(v == 0) # Força a variável a 0 (Falso)

# R7
for d in dia:
    for p in periodo:

        for idx_sala in range(len(salas)):

```

```

# CORREÇÃO: ler "sala" em vez de "tipo"
nome_sala_atual = str(salas.loc[idx_sala, "sala"]).strip()
capacidade_maxima = int(salas.loc[idx_sala, "quantidade"])

# Lista para guardar as variáveis das disciplinas que usam
ESTA sala, NESTE tempo
aulas_a_decorrer_neste_tipo_de_sala = []

for disci in range(len(disciplinas)):
    # 1. Ver qual a sala que esta disciplina exige
    sala_disci = str(disciplinas.loc[disci,
"sala_especial"]).strip()

    # Se for vazio, assume "Sala Normal" (nome exato que
está no salas.csv)
    if not sala_disci or sala_disci.lower() == "nan":
        sala_disci = "Sala Normal"

    # 2. Se a disciplina precisar DESTA sala (ex:
"Laboratório" == "Laboratório")
    if sala_disci == nome_sala_atual:
        disciplina_nome = disciplinas.loc[disci,
"disciplina"]

        for t in turma:
            v = variaveis[(t, disciplina_nome, d, p)]
            aulas_a_decorrer_neste_tipo_de_sala.append(v)

    if len(aulas_a_decorrer_neste_tipo_de_sala) > 0:
        # A soma das variáveis (1 ou 0) não pode exceder o
número de salas
        modelo.Add(sum(aulas_a_decorrer_neste_tipo_de_sala) <=
capacidade_maxima)
        #otimizacao
        buracos_totais = []

        max_p = max(periodo)
        valor_infinito = max_p + 1

        for prof, discs in disciplinas_por_professor.items():
            for d in dia:
                presencas_no_dia = []
                valores_para_maximo = []
                valores_para_minimo = []

                for p in periodo:
                    aulas_neste_tempo = []
                    for disc in discs:
                        for t in turma:
                            aulas_neste_tempo.append(variaveis[(t, disc, d,
p)])

                presenca = modelo.NewIntVar(0, 1, f"pres_{prof}_{d}_{p}")
                modelo.Add(presenca == sum(aulas_neste_tempo))
                presencas_no_dia.append(presenca)

```

```

        val_max = modelo.NewIntVar(0, max_p, "")
        modelo.Add(val_max == presenca * p)
        valores_para_maximo.append(val_max)

        val_min = modelo.NewIntVar(1, valor_infinito, "")
        modelo.Add(val_min == (presenca * p) + ((1 - presenca) *
valor_infinito))
        valores_para_minimo.append(val_min)

        total_aulas_dia = sum(presencas_no_dia)

        ultimo_tempo = modelo.NewIntVar(0, max_p, f"ult_{prof}_{d}")
        modelo.AddMaxEquality(ultimo_tempo, valores_para_maximo)

        primeiro_tempo = modelo.NewIntVar(1, valor_infinito,
f"prim_{prof}_{d}")
        modelo.AddMinEquality(primeiro_tempo, valores_para_minimo)

        tem_aula_dia = modelo.NewIntVar(0, 1,
f"foi_a_escola_{prof}_{d}")
        modelo.Add(total_aulas_dia > 0).OnlyEnforceIf(tem_aula_dia)
        modelo.Add(total_aulas_dia ==
0).OnlyEnforceIf(tem_aula_dia.Not())

        buracos_dia = modelo.NewIntVar(0, max_p, f"buracos_{prof}_{d}")

        modelo.Add(buracos_dia == (ultimo_tempo - primeiro_tempo + 1) -
total_aulas_dia).OnlyEnforceIf(tem_aula_dia)
        modelo.Add(buracos_dia == 0).OnlyEnforceIf(tem_aula_dia.Not())

        buracos_totais.append(buracos_dia)

    penalizacoes_ultimo_tempo = []
    ultimo_periodo = max(periodo)

    for t in turma:
        for c in range(len(disciplinas)):
            disc = disciplinas["disciplina"][c]
            for d in dia:
                v = variaveis[(t, disc, d, ultimo_periodo)]
                penalizacoes_ultimo_tempo.append(v)

    return disciplinas_por_professor , buracos_totais,
penalizacoes_ultimo_tempo

```

Função gerar_horario()

A função `gerar_horario()` é responsável por **construir, otimizar e validar um horário**. O processo é feito em várias etapas:

1. Preparação do modelo

É chamada a função `preparar_modelo()`, que:

- lê os ficheiros CSV;
- cria o modelo CP-SAT;
- cria todas as variáveis de decisão;
- aplica as regras R1-R7;
- calcula as variáveis associadas aos buracos dos professores.

No final, o modelo está pronto para ser resolvido.

2. Utilização de um horário anterior

Quando existe um `horario_base`, como acontece na construção de H1 a partir de H0, o horário anterior é usado como referência.

Para cada variável é comparado o valor antigo com o novo:

$$mudou = \begin{cases} 1 - var_atual, & \text{se o valor antigo era 1} \\ var_atual, & \text{se o valor antigo era 0} \end{cases}$$

Assim, `mudou = 1` significa que aquela aula foi alterada.

3. Penalização das alterações

Cada alteração recebe um peso elevado:

`PESO_MUDANCA = 100`

Enquanto os buracos recebem:

`PESO_BURACO = 1`

E as aulas ao fim do dia:

`PESO_ULTIMAO_TEMPO = 5`

O objetivo é:

$$\text{Objetivo} = 100 \times \text{alterações} + 1 \times \text{buracos} + 5 \times \text{aulas no ultimo periodo}$$

Isto faz com que **manter o horário anterior seja muito mais importante do que deixar o ultimo tempop livre ou reduzir um pequeno número de buracos.**

4. Resolução

O `CpSolver` procura uma solução que:

- cumpra todas as regras;
- minimize o número de alterações, quando existe horário anterior;

- minimize os buracos dos professores.

Por isso, no caso de H1, o solver não começa simplesmente do zero: tenta encontrar uma solução **o mais próxima possível de H0**.

5. Guardar a solução

Depois de o solver encontrar uma solução, os valores das variáveis são copiados para `horario_resultado`.

Assim, cada chave:

`(turma, disciplina, dia, periodo)`

fica associada ao valor:

0 → não há aula

1 → há aula

Este dicionário pode depois ser utilizado para comparar horários ou para construir o próximo horário incremental.

6. Impressão e validação

Por fim, o horário é apresentado para as turmas e professores e é passado ao `validar_horario()`.

Desta forma, existem duas verificações:

Solver → constrói uma solução válida

Validador → confirma as regras após a construção

Assim, o sistema privilegia a **estabilidade do horário**, evitando reconstruí-lo completamente sempre que os dados mudam.

```
In [ ]: def preparar_modelo(pasta_dados):
    """
    Lê os ficheiros de uma pasta específica, inicializa o motor matemático,
    cria as variáveis de decisão e aplica todas as regras (R1-R7 +
    Otimização).
    Devolve o modelo pronto a ser resolvido.
    """
    # 1. Carregar os ficheiros
    disciplinas = pd.read_csv(f"{pasta_dados}/disciplinas.csv")
    disponibilidade = pd.read_csv(f"
{pasta_dados}/disponibilidade_excecoes.csv")
    salas = pd.read_csv(f"{pasta_dados}/salas.csv")
    turmas = pd.read_csv(f"{pasta_dados}/turmas.csv")

    # 2. Definir constantes da semana
    dia = ["Seg", "Ter", "Qua", "Qui", "Sex"]
    periodo = [1, 2, 3, 4, 5]
```

```

turmas_lista = turmas["turma"].tolist()

# 3. Inicializar Modelo e Variáveis
modelo = cp_model.CpModel()
variaveis = cria_variaveis(modelo, turmas_lista, disciplinas, dia,
periodo)

# 4. Aplicar todas as regras
profs, buracos, penalizacoes_ultimo_tempo = aplica_regras(
    variaveis, turmas_lista, dia, periodo, turmas,
    salas, disciplinas, disponibilidade, modelo
)

# Devolver os pacotes de dados necessários para o Solver e para
imprimir
return modelo, variaveis, buracos, profs, disciplinas, turmas_lista,
dia, periodo, penalizacoes_ultimo_tempo

def gerar_horario(pasta_dados, horario_base=None):
    print(f"\nA preparar o modelo com dados da pasta '{pasta_dados}'...")

    modelo, variaveis, buracos, profs, disciplinas, turmas_lista, dia,
    periodo, penalizacoes_ultimo_tempo = preparar_modelo(pasta_dados)

    penalizacoes_mudanca = []

    # Se for passado um horário base (Fase Incremental - H1, H2, etc)
    if horario_base is not None:

        for chave, valor_antigo in horario_base.items():
            if chave in variaveis:
                var_atual = variaveis[chave]
                modelo.AddHint(var_atual, valor_antigo)

                mudou = modelo.NewIntVar(0, 1, f"mudou_{chave}")
                if valor_antigo == 1:
                    modelo.Add(mudou == 1 - var_atual)
                else:
                    modelo.Add(mudou == var_atual)
                penalizacoes_mudanca.append(mudou)

    # Definição do Objetivo
    PESO_MUDANCA = 100
    PESO_BURACO = 1
    PESO_ULTIMO_TEMPO = 5

    objetivo = (
        sum(buracos) * PESO_BURACO
        + sum(penalizacoes_ultimo_tempo) * PESO_ULTIMO_TEMPO
    )
    if penalizacoes_mudanca:
        objetivo += sum(penalizacoes_mudanca) * PESO_MUDANCA

    modelo.Minimize(objetivo)

```

```

solver = cp_model.CpSolver()
inicio = time.time()
res = solver.Solve(modelo)
fim = time.time()

horario_resultado = {}
tempo_execucao = fim - inicio
total_aulas_alteradas = solver.Value(sum(penalizacoes_mudanca)) / 2

if res == cp_model.OPTIMAL or res == cp_model.FEASIBLE: # sugerido por
LLM
    total_buracos = int(solver.Value(sum(buracos)))
    total_ultimo_tempo =
int(solver.Value(sum(penalizacoes_ultimo_tempo)))
    print(f"Horário gerado com sucesso em {tempo_execucao:.2f}
segundos!")
    print(f"Total de buracos: {total_buracos}")
    print(f"Aulas no último período: {total_ultimo_tempo}")

    horario_resultado = {}

    for chave, var in variaveis.items():
        horario_resultado[chave] = solver.Value(var)

    if penalizacoes_mudanca:
        print(f"Total de aulas alteradas face ao horário original:
{int(total_aulas_alteradas)}")

    texto_turmas = imprimir_horarios(
        solver,
        variaveis,
        turmas_lista,
        dia,
        periodo,
        disciplinas
    )

    texto_professores = imprimir_horarios_professores(
        solver,
        variaveis,
        turmas_lista,
        dia,
        periodo,
        profs
    )

    print("VALIDAÇÃO DO HORÁRIO")

    erros = validar_horario(
        horario_resultado,
        pasta_dados=pasta_dados
    )

    if not erros:

```

```

        print("0 horário respeita todas as regras (R1-R8)!")
    else:
        print("0 horário gerado apresenta violações:")
        for _erro in erros:
            print(f"    - {_erro}")

    return (
        horario_resultado,
        tempo_execucao,
        total_aulas_alteradas,
        total_buracos,
        total_ultimo_tempo,
        texto_turmas,
        texto_professores
    )

```

Impressão dos resultados

As funções de impressão apresentam o horário final de forma organizada, mostrando as aulas **por turma e por professor**.

Servem apenas para **visualizar a solução encontrada pelo solver**, sem alterar o modelo ou as regras.

```

In [ ]: # LLM inicio
def imprimir_horarios(solver, variaveis, turma, dia, periodo, disciplinas):
    texto = ""

    largura_col = 20

    for t in turma:
        texto += "\n"
        texto += "=" * 125 + "\n"
        texto += f"  HORÁRIO DA TURMA: {t}\n"
        texto += "=" * 125 + "\n"

        cabecalho = f"{'Período':<10}"
        for d in dia:
            cabecalho += f"| {d:<{largura_col}}"

        texto += cabecalho + "\n"
        texto += "-" * len(cabecalho) + "\n"

        for p in periodo:
            linha = f"{p:<10}"

            for d in dia:
                aula_neste_tempo = "---"

                for c in range(len(disciplinas)):
                    disciplina_nome = str(disciplinas["disciplina"][c])

                    v = variaveis[(t, disciplina_nome, d, p)]

```

```

        if solver.Value(v) == 1:
            aula_neste_tempo = disciplina_nome
            break

        linha += f"| {aula_neste_tempo:<{largura_col}}|"

    texto += linha + "\n"

    texto += "\n"

    return texto

def imprimir_horarios_professores(
    solver,
    variaveis,
    turma,
    dia,
    periodo,
    disciplinas_por_professor
):
    texto = ""

    largura_col = 25

    for prof, discs in disciplinas_por_professor.items():
        texto += "\n"
        texto += "=" * 125 + "\n"
        texto += f" HORÁRIO DO PROFESSOR: {prof}\n"
        texto += "=" * 125 + "\n"

        cabecalho = f"{'Período':<10}"
        for d in dia:
            cabecalho += f"| {d:<{largura_col}}|"

        texto += cabecalho + "\n"
        texto += "-" * len(cabecalho) + "\n"

        for p in periodo:
            linha = f"{p:<10}"

            for d in dia:
                aula_neste_tempo = "---"

                for disc in discs:
                    for t in turma:
                        v = variaveis[(t, disc, d, p)]

                        if solver.Value(v) == 1:
                            aula_neste_tempo = f"{t} ({disc})"
                            break

                if aula_neste_tempo != "---":
                    break

            linha += f"| {aula_neste_tempo:<{largura_col}}|"

```

```

        texto += linha + "\n"

    texto += "\n"

    return texto
# LLM Fim

```

Função `validar_horario()`

A função `validar_horario()` serve para **confirmar se o horário gerado pelo solver respeita todas as regras definidas**.

Para isso, lê novamente os dados dos ficheiros CSV e analisa o horário produzido, verificando as regras **R1 a R8**.

A função não cria um novo modelo nem executa o solver. Apenas **analisa a solução já obtida** e regista numa lista todas as possíveis violações encontradas.

No final:

- se a lista estiver vazia, o horário é considerado válido;
- caso existam elementos, são apresentados os erros encontrados e a regra que foi violada.

Assim, funciona como uma **segunda verificação independente** da solução produzida pelo solver.

```

In [ ]: # LLM inicio
def validar_horario(horario, pasta_dados="dados"):
    """
    Verifica um horário já produzido pelo solver.

    Não cria outro modelo e não executa Solve().
    Apenas lê os valores presentes em 'horario'.
    """

    erros = []

    # =====
    # Ler os dados dos CSV
    # =====

    disciplinas = pd.read_csv(
        f"{pasta_dados}/disciplinas.csv"
    )

    disponibilidade = pd.read_csv(
        f"{pasta_dados}/disponibilidade_excecoes.csv"
    )

```

```

salas = pd.read_csv(
    f"{pasta_dados}/salas.csv"
)

turmas_dados = pd.read_csv(
    f"{pasta_dados}/turmas.csv"
)

dias = ["Seg", "Ter", "Qua", "Qui", "Sex"]
periodos = [1, 2, 3, 4, 5]

turmas_lista = turmas_dados["turma"].tolist()

# R1
for t in turmas_lista:
    for d in dias:
        for p in periodos:

            total = 0
            for _, linha in disciplinas.iterrows():
                nome = str(linha["disciplina"])
                total += horario.get((t, nome, d, p), 0)

            if total > 1:
                erros.append(f"R1: {t} tem {total} aulas "f"em
simultâneo em {d}/{p}")

# R2, R3 e R4
for t in turmas_lista:
    for _, linha in disciplinas.iterrows():
        nome = str(linha["disciplina"])
        carga = int(linha["carga_semanal"])

        duplo = ( str(linha["duplo_periodo"]).strip().lower() == "sim")

        aulas = []
        for d in dias:
            for p in periodos:
                if horario.get((t, nome, d, p), 0) == 1:
                    aulas.append((d, p))

        # R2
        if len(aulas) != carga:
            erros.append(f"R2: {t}/{nome} tem "{len(aulas)} aulas;
"f"esperado {carga}")

        # R3 e R4
        for d in dias:
            ps = sorted(p for dia_aula, p in aulas if dia_aula == d)

            # R3
            if not duplo:
                if len(ps) > 1:
                    erros.append( f"R3: {t}/{nome} aparece "f"mais de

```

```

uma vez em {d}")

        # R4
        else:
            if ps:
                bloco_valido = ( len(ps) == 2 and ps[1] == ps[0] +
1)

                if not bloco_valido:
                    erros.append(f"R4: {t}/{nome} "f"não forma
bloco duplo "f"válido em {d}: {ps}")

# R5
# Um professor não pode dar duas aulas simultâneas

professores_unicos = disciplinas["professor"].unique().tolist()

for professor in professores_unicos:

    linhas_do_prof = disciplinas[disciplinas["professor"] ==
professor]
    nomes_disciplinas_do_prof =
linhas_do_prof["disciplina"].tolist()

    for d in dias:
        for p in periodos:
            total_aulas_prof = 0

            for t in turmas_lista:
                for nome in nomes_disciplinas_do_prof:

                    total_aulas_prof += horario.get((t, nome, d,
p),0)

                    if total_aulas_prof > 1:
                        erros.append(f"R5: {professor} tem
{total_aulas_prof} aulas " f"em simultâneo em {d}/{p}")

# R6
# Professor não pode dar aula quando indisponível

for _, execucao in disponibilidade.iterrows():

    professor_indisponivel = str(execucao["professor"]).strip()
    d = str(execucao["dia"]).strip()
    p = int(execucao["periodo"])

    for t in turmas_lista:
        for _, linha in disciplinas.iterrows():
            nome = str(linha["disciplina"])
            professor = str(linha["professor"]).strip()

            if (professor== professor_indisponivel):
                if horario.get((t, nome, d, p),0) == 1:

```

```

        erros.append(f"R6: {professor} tem " f"aula de
{nome} em " f"{d}/{p}, mas está " f"indisponível" )

# R7
# Capacidade das salas

# Lemos a coluna 'sala' para que as chaves fiquem:
# {"Sala Normal": 6, "Laboratório": 1, "Ginásio": 1}
capacidade = {
    str(linha["sala"]).strip(): int(linha["quantidade"])
    for _, linha in salas.iterrows()
}

for d in dias:
    for p in periodos:
        usadas = {}

        for t in turmas_lista:
            for _, linha in disciplinas.iterrows():
                nome = str(linha["disciplina"])

                if horario.get((t, nome, d, p), 0) != 1:
                    continue

                sala_necessaria =
str(linha["sala_especial"]).strip()

                # Se não tiver nada, mapeamos para "Sala Normal"
                (nome exato no salas.csv)
                if (not sala_necessaria or sala_necessaria.lower()
== "nan"):
                    sala_necessaria = "Sala Normal"

                usadas[sala_necessaria] =
usadas.get(sala_necessaria, 0) + 1

                for sala_req, quantidade in usadas.items():
                    if sala_req not in capacidade:
                        erros.append(
                            f"R7: não existe capacidade para a sala
'{sala_req}'"
                        )
                    elif quantidade > capacidade[sala_req]:
                        erros.append(
                            f"R7: existem {quantidade} aulas para a sala "
                            f"'{sala_req}', mas só existem
{capacidade[sala_req]}"
                        )

# R8
# Confirmar a estrutura dos CSV

obrigatorias = {

    "turmas": {
        "turma"

```

```

    },

    "disciplinas": {
        "disciplina",
        "professor",
        "carga_semanal",
        "duplo_perodo",
        "sala_especial"
    },

    "salas": {
        "tipo",
        "quantidade"
    },

    "disponibilidade": {
        "professor",
        "dia",
        "periodo"
    }
}

tabelas = {
    "turmas": turmas_dados,
    "disciplinas": disciplinas,
    "salas": salas,
    "disponibilidade": disponibilidade}

for nome_tabela, colunas in obrigatorias.items():
    if not colunas.issubset(
        tabelas[nome_tabela].columns
    ):

        erros.append(
            f"R8: o CSV de {nome_tabela} "
            f"não tem as colunas necessárias"
        )

return erros
# LLM fim

```

Horarios

```

In [ ]: (
    horario_h0,
    tempo_h0,
    alteracoes_h0,
    buracos_h0,
    ultimo_h0,
    texto_h0_turmas,
    texto_h0_professores
) = gerar_horario("dados")

(

```

```

    horario_h1,
    tempo_h1,
    alteracoes_h1,
    buracos_h1,
    ultimo_h1,
    texto_h1_turmas,
    texto_h1_professores
) = gerar_horario(
    "dados_v2"
)

(
    horario_h1_h0,
    tempo_h1_h0,
    alteracoes_h1_h0,
    buracos_h1_h0,
    ultimo_h1_h0,
    texto_h1_h0_turmas,
    texto_h1_h0_professores
) = gerar_horario(
    "dados_v2",
    horario_base=horario_h0
)

(
    horario_h2,
    tempo_h2,
    alteracoes_h2,
    buracos_h2,
    ultimo_h2,
    texto_h2_turmas,
    texto_h2_professores
) = gerar_horario(
    "dados_v3"
)
# LLM Inicio

mo.vstack([
    mo.md("# H0 - Dados_v1"),
    mo.md(f"""
**Tempo:** {tempo_h0:.2f} s
**Buracos dos professores:** {buracos_h0}
**Aulas no último período:** {ultimo_h0}
"""),
    mo.plain_text(texto_h0_turmas),
    mo.plain_text(texto_h0_professores),

    mo.md("# H1 - Dados_v2"),
    mo.md(f"""
**Tempo:** {tempo_h1:.2f} s
**Buracos dos professores:** {buracos_h1}
**Aulas no último período:** {ultimo_h1}
"""),
    mo.plain_text(texto_h1_turmas),
    mo.plain_text(texto_h1_professores),

```

```

    mo.md("# H1 apartir de H0 – Dados_v2"),
    mo.md(f"""
**Tempo:** {tempo_h1_h0:.2f} s
**Alterações face ao H0:** {int(alteracoes_h1_h0)}
**Buracos dos professores:** {buracos_h1_h0}
**Aulas no último período:** {ultimo_h1_h0}
"""),
    mo.plain_text(texto_h1_h0_turmas),
    mo.plain_text(texto_h1_h0_professores),

    mo.md("# H2 – Dados_v3"),
    mo.md(f"""
**Tempo:** {tempo_h2:.2f} s
**Buracos dos professores:** {buracos_h2}
**Aulas no último período:** {ultimo_h2}
"""),
    mo.plain_text(texto_h2_turmas),
    mo.plain_text(texto_h2_professores),
])
# LLM Fim

```

Uso de LLM

Como material de apoio à resolução deste trabalho, foram usados os LLMs ChatGPT e Google Gemini. Os LLMs ChatGPT e Google Gemini geraram por completo as secções **"Impressão de Resultados"**, a impressão das métricas da resolução e a função **validar_horario()**, ajudaram no debug das mesmas e geraram toda a explicação em Markdown presente no notebook, mas contudo, o grupo garantiu a supervisão durante todo o processo. Para além disso, ambos os LLMs serviram de grande ajuda em termos de debug, avaliação de estratégias e ferramentas a utilizar. Os chats abaixo contêm prompts não relacionados, mas que cobrem o que foi referido acima. [ChatGPT 1](#) [ChatGPT 2](#) [Google Gemini 1](#) [Google Gemini 2](#) [Google Gemini 3](#) O grupo declara-se completamente responsável por todo código apresentado independentemente da origem

In []: